# COM10393 – MÉTODOS DE OTIMIZAÇÃO

## PROVA III

**Objetivo:** implementar heurísticas de refinamento (buscas locais) para o problema *p-hub* Centro Não Capacitado de Múltipla Alocação (UMApHCP).

Geralmente, este problema é encontrado no planejamento de sistemas de distribuição envolvendo, por exemplo, produtos perecíveis ou produtos sensíveis ao fator tempo, como cargas urgentes. Aplicações podem ser encontradas em redes de telecomunicações, linhas aéreas, correios, transportes em geral, etc.

Nesses sistemas, produtos devem ser distribuídos entre "nós" (clientes, cidades, armazéns, equipamentos, etc.) com o menor custo possível. Assume-se que todo nó deve ser atendido por pelo menos um *hub*, e o transporte direto entre nós que não são *hubs* não é permitido. Por fim, apenas $p$ *hubs* devem ser instalados.

O objetivo do UMApHCP consiste em determinar quais nós serão *hubs* e definir o custo de transporte entre todos os pares de nós, passando por pelo menos um *hub*, de modo que o maior custo de transporte entre todos os pares seja minimizado.

O UMApHCP pode ser representado por meio de um grafo $G = (N, A)$ em que $N$ é um conjunto com $n$ nós e $A = \{(i, j) \mid i, j \in N\}$ representa o conjunto de pares de nós. Todo nó $i \in N$ pode ser um *hub*, porém, deve-se escolher apenas $p$ nós para serem *hubs*. Para cada par de nós $(i, j) \in A$, existe um custo $c_{ij}$, que pode ser definido como distância, tempo, valor financeiro, etc.

Não é permitido o transporte direto entre nós não *hubs*. Então, a distribuição entre dois nós não *hubs* $i$ e $j$ deve passar por pelo menos um *hub* ($k = l$). Logo, o custo de transporte entre quaisquer nós $i$ e $j$ (*hubs* ou não *hubs*) passando pelos *hubs* $k \in N$ e $l \in N$ é dado por:

$$t_{ij} = \beta c_{ik} + \alpha c_{kl} + \lambda c_{lj}$$

$c_{ik}$ é o custo de coleta entre o nó de origem $i$ e o *hub* $k$; $c_{kl}$ representa o custo de transferência entre o *hub* $k$ e o *hub* $l$; e $c_{lj}$ representa o custo de distribuição entre o *hub* $l$ e o nó de destino $j$.

A esses custos, são associados os fatores $\beta$, $\alpha$ e $\lambda$, que "nivelam" os valores. Por exemplo, o custo entre *hubs* tende a ser menor, pois representa a economia de escala devido à concentração de fluxo nos *hubs*.

A função objetivo do UMApHCP pode ser definida conforme a expressão abaixo, ou seja, dada uma solução $s$, a sua função objetivo indica o valor do maior custo de transporte possível entre todos os pares.

$$f(s) = \max t_{ij} \quad \forall\, i, j \in N$$

Logo, deve-se determinar quais serão os $p$ nós selecionados como *hubs*, a partir dos quais o custo de transporte entre todos os pares $(i, j)$ deve ser calculado. Assim, o valor do maior custo de transporte será a função objetivo, que deverá ser minimizada.

A Figura 1 apresenta a solução ótima para uma instância com 20 nós e 4 *hubs*. Nessa figura, os nós selecionados como *hubs* estão destacados em vermelho, e o maior custo de transporte é entre os nós 0 e 19, passando pelos *hubs* 5 e 13.

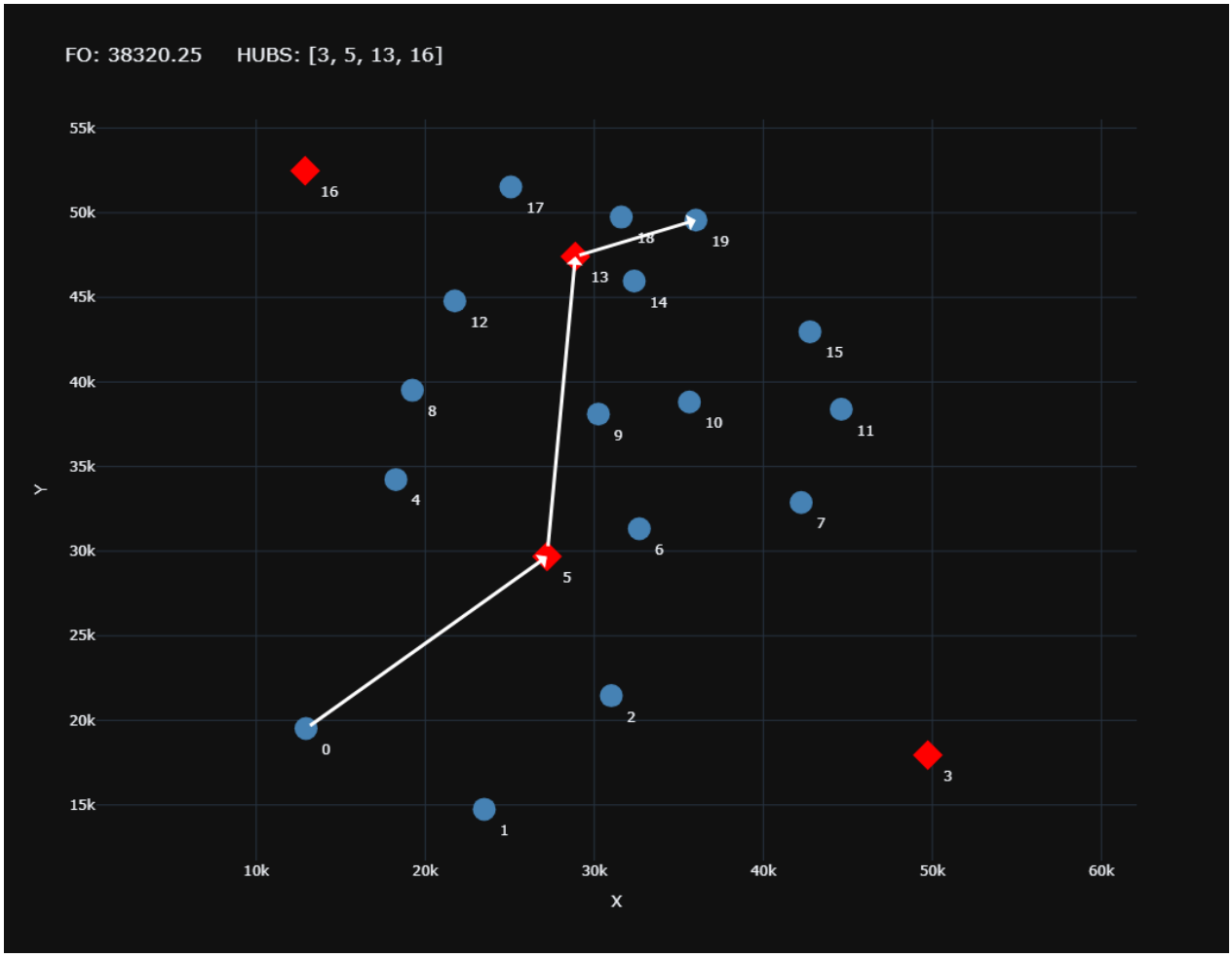



Figura 1 – Solução ótima para 20 nós e 4 *hubs*.

Para que uma solução seja considerada viável, deve-se garantir que exatamente $p$ nós devem ser selecionados como *hubs*, e os caminhos para distribuição entre todos os pares $(i, j)$ devem passar, obrigatoriamente, por pelo menos um nó *hub*.

O custo $c_{ij}$ entre cada par $(i, j) \in A$ deve ser calculado pela distância euclidiana entre os nós, ou seja:

$$c_{ij} = \sqrt{(x_i - x_j)^2 - (y_i - y_j)^2}$$

Assim, o problema é simétrico, ou seja, $c_{ij} = c_{ji}$. Os fatores de custo devem ser $\beta = 1$, $\alpha = 0{,}75$ e $\lambda = 1$, indicando uma "economia" de 25% no custo de transporte entre *hubs*.

Em anexo, está um código em C++ que deve ser utilizado para resolução desta prova. Também em anexo estão 4 instâncias para teste, cujo formato é apresentado abaixo.

| | | |
|---|---|---|
| <*n*> | <p> | número de nós número de hubs |
| <*x*[1]> <*y*[1]> | | coordenadas *x* e *y* do nó 1 |
| : | | |
| : | | |
| <*x*[*n*]> <*y*[*n*]> | | coordenadas *x* e *y* do nó *n* |

Figura 2 – Formato das instâncias.

No código, já estão definidas as variáveis com os dados de entrada e uma estrutura para representar uma solução do problema, que é formada por um vetor com $n$ posições, sendo as $p$ primeiras indicando os *hubs* (id dos nós *hubs*), e as demais $(n - p)$ referentes aos nós não *hubs*. Também são definidas algumas funções básicas para o tratamento do problema.

Utilizando o código, **nas áreas indicadas**, resolva as questões a seguir.

**Questão 1** (0,7): Faça um método com uma busca local **randômica**.

**Questão 2** (0,8): Faça um método com uma busca local **de primeira melhora**.

**Questão 3** (1,0): Faça um método com uma busca local de **melhor melhora**.

**Observações:**

- Todos os métodos devem receber uma solução a ser "refinada" por parâmetro (referência), e não devem possuir "retorno" (void).

- Os arquivos com o código fonte devem ser renomeados com o nome e sobrenome (ex: geraldo_mauri.cpp e geraldo_mauri.h).

- A tabela a seguir apresenta o valor da melhor solução conhecida para cada instância.

| Instância | Melhor FO |
|---|---|
| 1 | 24574,90 |
| 2 | 38320,25 |
| 3 | 50707,87 |
| 4 | 51860,03 |